

FEDERAL STATE AUTONOMOUS  
Educational Institute  
HIGHER EDUCATION  
"NATIONAL RESEARCH UNIVERSITY  
Higher School of Economics  
Faculty of Informatics, Mathematics and Computer Science

**Trenikhin Stanislav Georgievich**

Final qualifying work – MASTER'S THESIS  
in the field of study Artificial Intelligence and Computer Vision

**"VISION-LANGUAGE-ACTION MODELS FOR ROBOTICS TASKS"**

Reviewer

Senior Research Engineer.  
Mathematical Robotics department,  
NTU Sirius – Sirius

**Maxim Grigorov**

Supervisor

PhD/ Doctor of Science, Full  
Professor. Information Systems and  
Technologies Department, HSE  
University – Nizhny Novgorod

**Andrey Savchenko**

Supervisor

PhD,  
Associate Professor. Department of Data  
Analysis and Artificial Intelligence, HSE  
University – Moscow

**Ilya Makarov**

## Content

|                                                                                                                      |    |
|----------------------------------------------------------------------------------------------------------------------|----|
| Introduction. ....                                                                                                   | 4  |
| 1. Theoretical foundations of the Vision-Language-Action models. ....                                                | 5  |
| 1.1. Evolution of control systems in robotics. ....                                                                  | 5  |
| 1.1.1. The concept of multimodality: combining vision, language,<br>and action. ....                                 | 7  |
| 1.1.2. Overview of the VLA (Vision-Language-Action) paradigm. ....                                                   | 9  |
| 1.2. SmolVLA model architecture ....                                                                                 | 11 |
| 1.2.1. Multi-level hybrid architecture: inter-level transformations<br>and feature hierarchy ....                    | 11 |
| 1.2.2. Multimodal context synthesis: integration of proprioception and<br>semantics into a uniform latent space .... | 13 |
| 1.2.3. Specialized Action-Expert and RTC: Flow Matching and<br>high-frequency trajectory synthesis. ....             | 14 |
| 1.2.4. Strategies for overcoming the perceptual limitations<br>of standard ViT architectures. ....                   | 16 |
| 1.3. Learning from demonstrations (Imitation Learning) ....                                                          | 17 |
| 1.3.1. Imitation learning algorithms: ACT (Action Chunking Transformer)<br>and Diffusion Policy. ....                | 17 |
| 1.3.2. The problem of generalization in the open world. ....                                                         | 19 |
| 1.3.3. Features of collecting datasets for VLA tasks. ....                                                           | 21 |

|                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------|----|
| 2. Practical implementation and experiments in the LeRobot environment. ....                        | 23 |
| 2.1. LeRobot Software ecosystem. ....                                                               | 23 |
| 2.1.1. Overview of the LeRobot Framework by Hugging Face. ....                                      | 23 |
| 2.1.2. Environment configuration and SmolVLA integration. ....                                      | 25 |
| 2.1.3. Preparation and normalization of data (LeRobotDataset). ....                                 | 26 |
| 2.2. Setting up the experiment. ....                                                                | 29 |
| 2.2.1. Hardware setup and robot configuration (Leader-Follower setup) . ....                        | 31 |
| 2.2.2. Data collection protocol and demonstration recording methodology. ..                         | 34 |
| 2.2.3. Methodology and program regulations for model fine-tuning. ....                              | 36 |
| 2.3. Analysis of results. ....                                                                      | 40 |
| 2.3.1. Metrics (Success Rate). ....                                                                 | 40 |
| 2.3.2. Evaluation of the stability of the quantized model for embedded<br>processors with NPU. .... | 42 |
| Conclusion. ....                                                                                    | 44 |
| References . ....                                                                                   | 46 |

## **Introduction.**

Modern robotics and artificial intelligence systems are witnessing a fundamental shift from classical isolated control systems to end-to-end multimodal Vision-Language-Action (VLA) policies. The integration of large pre-trained general-purpose foundational models enables robotic systems to directly interpret unstructured verbal user instructions and sensor data (open world) into spatiotemporal trajectories. However, direct deployment of monolithic VLA policies on autonomous robots faces a critical infrastructure barrier: the computational overhead and memory bandwidth limitations of embedded devices.

Behavior cloning and the creation of multimodal AI policies are being explored in depth in the open-source community, including the Hugging Face Robotics research group (LeRobot ecosystem). However, the vast majority of existing approaches rely on heavy server computing clusters.

The objective of this work is to study the VLA model and ways to improve the accuracy and stability of a system based on the SmolVLA (0.45B) low-parameter transformer and an actuator robotic manipulator in an unstructured environment. To achieve this goal, it is necessary to deploy the LeRobot framework, perform a layer-by-layer study of the model, verify the parameter utilization rate, conduct field tests and record the obtained measurements, explore ways to improve accuracy, and explore the application of the model to embedded devices.

## **1. Theoretical foundations of the Vision-Language-Action models.**

### **1.1. Evolution of control systems in robotics.**

The evolution of robot control systems has evolved from rigidly deterministic mathematical models to flexible neural network architectures. Beginning with punch cards, first used in 1804 by the French inventor Joseph Marie Jacquard, who created the weaving loom, the idea of CNC machines used in modern industry was formalized in 1952 with the first working prototype of the Cincinnati Milacron milling machine. Subsequently, a special motion control language known to us as “G-code” (a sequence of trajectories at a given velocity) emerged, was established as a standard in the 1960s by the EIA (Electronic Industries Alliance) under the name RS-274, and later became the international standard ISO 6983, which is still used to this day in the vast majority of applications, from 3D printing and milling with 5-DOF, to manipulators with 6 or more DOF.

The control of the actuators was based on rigorous mathematical calculations. The task execution process was broken down into isolated subtasks:

Path planning (generation of a collision-free path).

Inverse kinematics (calculation of the rotation angles of each node to reach a given point in space.

Execution of calculated movements by controllers at the level of stepper and servo drives.

Such systems operate effectively only in strictly structured conditions (for example, on a factory conveyor belt), and the slightest deviation, i.e., a shift of a target object by a couple of microns/millimeters/centimeters or a change in its shape, renders the algorithm ineffective, since it lacks the ability to adapt.

With the development of computer vision, the "Sense-Plan-Act" architecture emerged. This system, consisting of separate, independent modules:

Vision (neural network) finds an object and determines its 3D coordinates.

The planner calculates the path based on these coordinates.

The executor transmits commands to the motors.

However, if the vision module is off by a few pixels when determining depth, all subsequent calculations become incorrect (Error Propagation). Integrating such modules requires complex manual interface configuration.

End-to-end learning is a modern approach that abandons modularization in favor of end-to-end learning. In this paradigm, a single neural network is trained to directly map input sensory data (pixels from cameras) to output control commands (currents or motor angles). Because the model sees the results of its actions in every frame of the video stream, it can adjust movement on the fly, even if the object is shifting. Using the Vision-Language-Action architecture within end-to-end learning allows for the addition of textual context to this cycle, transforming the robot into a versatile agent capable of interpreting complex commands in an unpredictable environment.

In this study, the transition to an end-to-end control paradigm is examined using *the SO-101* physical manipulator as an example. The technology stack is based on *the LeRobot Hugging Face ecosystem* and *the SmolVLA (0.45B)* multimodal VLA model. Visual perception of the system is provided by a pair of cameras: an RGB-D *Intel RealSense D435i* and an RGB USB webcam. The *Intel RealSense D435i device* in addition to depth map construction hardware, is equipped with an inertial module (IMU), which expands the robot's proprioception capabilities. The work uses specialized utilities from the *pyrealsense2 standard library* for intrinsic calibration and on-hardware preprocessing. The experimental part of the study includes the Imitation learning stage.

### 1.1.1. The concept of multimodality: combining vision, language and action.

In the context of robotics, multimodality is the ability of a control system to process and integrate data from different types (modalities) into a single feature space. For VLA models, such as SmolVLA, three key components are visual, textual, and proprioceptive (data about the state of the robot itself).

The visual modality (Vision) is the primary source of information about the external environment. In multimodal models, **the ViT** [1] / **SigLIP** [2] visual encoder not only segments objects but also extracts their semantic features, enabling the robot to understand not only the geometric coordinates of an object but also its properties.

Text modality (Language) is high-level planning. The integration of language models, in this case **SmolLM** [3], brings common sense reasoning to robotics. Text serves as a link between the user's abstract task and concrete actions. Using language allows us to move away from rigid commands like `move_to (x, y, z)` to natural instructions like "clean the table." The model must decide for itself what exactly on the table is unnecessary, using its text knowledge accumulated through pre-training on large datasets.

The Action modality is proprioception and motor control. It includes current joint angles (joint states) and predicted trajectories. In a multimodal approach, action is viewed as another language. Motor commands are tokenized like words in a sentence, allowing the neural network to predict the next movement as naturally as the next word in a text.

The true power of multimodality is found in their intersection (The VLA Fusion):

Vision + Language allows the robot to identify objects by description ("pick up the glass from the table").

Language + Action ensures commands are carried out with nuance (the command "put it down gently" changes the dynamics of the movement compared to "drop it").

Vision + Action creates closed-loop control, meaning that visual changes in each subsequent frame instantly adjust the trajectory of the mechanism, in this case the manipulator.



### 1.1.2. Overview of the VLA paradigm.

The Vision-Language-Action (VLA) paradigm represents the culmination of the convergence of computer vision, natural language processing, and robotic control theory. Unlike classical systems, where each modality is processed in isolation, VLA models integrate them within a single computational graph, moving from discrete commands to semantic control.

While traditional G-code or manipulator APIs require a trajectory to be described as a set of coordinates, VLA models allow for semantic concepts. Instead of programming an L1, L2, L3 sequence to avoid an obstacle, the operator commands "move the glass without hitting the box." The model, using visual tokens and knowledge from the language core, independently synthesizes a safe trajectory.

*SmolVLA* is a model pretrained on 23,000 instances from approximately 500 community datasets [4]. The model doesn't learn to control from scratch, it already possesses knowledge of the world's physics and object properties, acquired through pretraining. For the specific *SO-101* robot, only fine-tuning is required to adapt the general knowledge to the specific kinematics and camera sensor. In this study, the author uses the Intel *RealSense D435i*.

This study highlights three critical advantages of the VLA paradigm:

Generalization: The ability to handle objects and lighting conditions that were not encountered in the training set.

Noise Resistance: Through end-to-end training, the system ignores visual clutter, focusing on semantically meaningful features.

Intuitive interaction: The robot can be controlled by a user without programming skills, through a natural language interface.

It's especially noteworthy that the evolution of control systems has led to the emergence of VLA models that overcome the limitations of the rigid algorithms of the past. The use of the compact *SmolVLA* architecture combined with the *LeRobot ecosystem* and *SO-101 manipulators*, opens up opportunities for a rapid "cold start"

in the development and deployment of affordable intelligent robotic systems.

## 1.2. SmolVLA model architecture.

### 1.2.1. Multi-level hybrid architecture: inter-level transformations and feature hierarchy.

The visual perception framework is based on *ViT-B*. The camera image is broken into 16x16 pixel patches, which are projected into 768-pixel vectors. Positional embedding with 1024 tokens allows the model to preserve spatial structure. Like the classic Vision Transformer implementation, the architecture under study utilizes learned positional embedding, with corresponding weights incorporated into the model structure. The model has a characteristic size of 86 million parameters and is a 12-layer encoder with 12 attention heads. The *SigLIP model used* was trained using Sigmoid as the logit normalization function, as opposed to the classic Softmax.

The cognitive multimodal core is a 252 million parameter *SmolLM2* (*lm\_expert*) with a vocabulary size of 49280, consists of 16 GQA layers of Grouped-Query Attention (GQA), significantly reducing the computational complexity and acts as a central hub, synthesizing three information flows. A text command forms the semantics of the task, and visual and proprioceptive features of the manipulator are transmitted along with linguistic data. Thus, Expert processes a single unified tensor, seamlessly linking NLP, vision, and proprioception, ignoring linguistic noise and focusing on physics. An internal expansion of layers to 2048 enables the computation of complex arcs and trajectories of the manipulator's motion. In fact, this gives the ***Flow Matching*** vector field [5].

Temporal synthesis of actions (Action Expert is the final stage, using ***Ordinary Differential Equations*** [6], another powerful feature of the architecture. Hidden features, "Keys" and "Values" from the VLM, are fed to the Expert stage. Also, a series of "action\_time\_mlp" of dimension 1440 is received, including the "noise" of positions (`torch.normal (mean=0.0, std=1.0 size=shape, dtype=torch.float32, device=device)`) 720 and sine-cosine temporal embedding 720, transforming the time and noise vectors into dimension 720. The incoming noise is

gradually generated by the model in a cycle step by step into a dynamic plan (action chunk) of dimension 32, each step of which is executed by the LeRobot physical controller, controlling the servo motors, from an approximate direction, refining at each iteration (by default, *num\_steps* = 10). At each step, the model accesses the VLM memory (dim=960) via Cross-Attention and refines the vector.

Thus, the output data of a relatively heavy VLM network is a vector field Flow Matching, fall into the lightweight block of simple differential equations ***RTC*** (*Real-Time Chunking*) [7], step by step and “seamlessly” generating smooth movements while the neural network generates a new vector field.

Structurally, the mechanism of all models can be described as follows:

*ViT (768 → 960): Extracting features of objects, their shape and texture.*

*Language (49 280 → 960): channel for receiving external settings.*

*State (32 → 960): angle data from servos.*

*VLM (960): generates a necessary action vector field to solve a problem.*

*Action (720): High-frequency computation of smooth physical trajectory.*

Having studied the SmolVLA architecture and discovered that the ViT/SigLIP architecture used as a vision encoder, which uses only three standard RGB channels, makes it impossible to transmit RGB-D data from the Intel RealSense camera to the model, measures will be taken in this work to improve the accuracy of spatial positioning. The author will consider the possibility of using binocular vision by transmitting independent streams from the left and right infrared sensors of the RealSense D435i camera, either by modifying the ViT architecture itself or by implementing an additional adapter. In this case, the SmolVLA architecture will be able to extract depth cues, or depth from parallax, directly in the latent space. This will allow the model to generate a more accurate spatial map without resorting to computationally expensive algorithms.

### **1.2.2. Multimodal context synthesis: integration of proprioception and semantics into a uniform latent space**

A main feature of the VLM cognitive core is the ability to manipulate data of different physical natures within a single mathematical representation. The process of forming a multimodal context is implemented through a mechanism of inter-layer alignment and early fusion.

The VLM includes a proprioceptive adaptation channel (Self-Perception), allowing the system to take into account the robot's current state. The robot's current joint angle vector (dim=32) is fed to the *state\_proj projector*, and a linear layer expands this data to 960 dimensions, making it compatible with the latent space of the language model. This is the robot's "muscle sense" without which precise movement correction is impossible.

After individual preparation of each modality, the VLM core receives a unified tensor as input, of the visual block (1024 tokens, dim=960), proprioceptive (dim=960), and language blocks (dim=960). Using a dimension of 960 for all streams ensures seamless attention.

The joint attention and inference mechanism is implemented by sixteen layers of the transformer, facilitating token interactions through the Self-Attention mechanism. In this unified space, text instructions act as a filter, directing the model to focus on specific "visual words" (objects), while proprioceptive data is correlated with the visual coordinates of the target, allowing the model to calculate the error signal between the current and desired positions. Synthesizing data in a single 960-dimensional latent space transforms disparate sensory inputs into a coherent picture of the world, allowing the system to control based not only on "vision" but also on "sensations" in the context of the task.

### 1.2.3. Specialized Action-Expert and RTC: Flow Matching and high-frequency trajectory synthesis.

While the cognitive core (VLM Core) is responsible for global planning and understanding of command semantics, the Action-Expert block acts as the system's "cerebellum." Its task is the high-speed generation of smooth and physically correct trajectories.

Latent features from the VLM core (dim=960) are translated into the 720-dimensional expert space. This bottleneck forces the model to ignore redundant linguistic context and focus exclusively on the spatial-dynamic characteristics of movement. Within each expert layer, an expansion of up proj features to 2048 is implemented. This provides sufficient mathematical power to approximate the nonlinear dependencies that arise as the robot moves in 3D space.

The final stage of the architecture is the transformation of high-level expert features into a stream of physical commands via a specialized Action Head. To ensure smooth movements, an *action\_time\_mlp block* with an input dimension of 1440 is used. It combines expert layer features (720) with a temporal embedding (720). This allows the model to implement the Flow Matching method [5], predicting not a static position, but a dynamic change in state (velocity vector field). Instead of step-by-step prediction, the model generates a predictive packet (Action Chunking) in a space of dimension 32. This packet contains a time planning horizon, which allows the physical controller, in this case a LeRobot SO-101, to smooth the trajectory and compensate for hardware delays in the servo drives (Feetech SCS3215). The final projection via action\_out\_proj translates the 720-dimensional latent features into a 32-dimensional output vector, which is subsequently interpreted as the angular coordinates of the joints and the gripper state of the manipulator.

Splitting the model into a heavyweight kernel (VLM) and a lightweight RTC enables asynchronous soft control. This is critical for closed-loop camera operation, as it allows the system to instantly correct the trajectory if the target object moves

accidentally.

Thus, the *Ordinary Differential Equations* unit sequentially transforms the vector field to soft movement. Combined with Flow Matching technology, SmolVLA overcomes the limitations of discrete control, providing the mechanism, in this case the SO-101 manipulator, with biologically similar smoothness and precision of movement. This hierarchical structure allows for the efficient distribution of computing resources, providing a deep understanding of the multimodal context combined with high frequency and precision of physical control, as confirmed by benchmarks on github.com. Thus, the authors of the work *SmolVLA: A vision-language-action model for affordable and efficient robotics* note :

*“In an effort to improve VLA's efficiency, TinyVLA (Wen et al., 2024) trained a lightweight sub-1B model from scratch on multimodal data and then fine-tuned it on robotics datasets, although the absence of large-scale pretraining on robotics data hinders wider generalization capabilities. SmolVLA shares similar goals with most of these efforts, aiming to develop and release open-source models that are both performant and highly efficient in terms of training and inference.” [4]*

#### 1.2.4. Strategies for overcoming the perceptual limitations of standard ViT architectures.

Despite the high semantic density of features extracted by the ViT-B/16 encoder, its standard three-channel architecture imposes limitations on the direct use of depth data from the Intel RealSense D435i. To address the problem of precise spatial positioning of the SO-101 manipulator and target object, this study proposes the following possible architectural solutions:

*Binocular Aggregation (Stereo Fusion)*: Instead of using a software-calculated depth map, the author proposes using raw data from the left and right infrared sensors. This allows the model to extract stereopsis (parallax) features directly in the latent space. Technically, this is implemented by feeding two independent visual streams into a shared multimodal context, where the Self-Attention mechanism in the VLM core calculates spatial displacements of objects.

*Architectural modification of the input stage*: We are considering expanding the dimensionality of the first convolutional layer (patch\_embedding) from 3 to 4 channels (RGB-D). This requires partial fine-tuning of the visual encoder, but allows for the integration of geometric data at the earliest stage of tokenization, which is critical for capturing small objects.

*Using Modal Adapters*: Implementing an Intermediate (Linear or Convolution) Bottleneck adapter), which converts depth data into 768-dimensional "geometric tokens" that are concatenated with visual tokens before entering the main kernel.

It is assumed that neural network extraction of parallax features is in any case more efficient than the construction of dense point clouds by classical algorithms, on the other hand, the use of depth maps calculated by RealSense hardware is “free”.



### **1.3. Learning from demonstrations (Imitation Learning).**

#### **1.3.1. Imitation learning algorithms: ACT (Action Chunking Transformer) and Diffusion Policy**

The current shift in robotic system learning from real-world demonstrations (Behavior Cloning) is associated with the transition from step-by-step action prediction (Step-by-Step Autoregression) to batch generation of predictive trajectories (Action Chunking). The two most representative architectural approaches in this class are ACT and Diffusion Policy.

The ACT (Action Chunking Transformer) [8] architecture is based on a combination of a variational autoencoder (Conditional VAE) [9] and a classical transformer [10]. At the inference stage, ACT receives current visual observations and a proprioception vector, generating a fixed-time packet of actions (chunk)  $T$  steps ahead at the output. The use of a latent variable C-VAE allows the model to simulate multimodal distributions (different ways of performing the same task). However, the main drawback of ACT is its high sensitivity to calibration quality and the “freezing” effect (compounding error / execution lag): the robot executes the current packet of actions, after which the loop freezes in anticipation of heavy inference of the next chunk, which leads to discrete, intermittent movements of the robot.

Unlike ACT, Diffusion Policy replaces the VAE-based generator with a denoising process. The model is trained to approximate noisy velocity fields and, in a fixed number of steps (e.g., from 100 to 16 denoising steps), transforms random Gaussian noise into a smooth spatiotemporal trajectory arc. The diffusion approach demonstrates unrivaled stability when avoiding dynamic obstacles and high positioning accuracy. However, Diffusion Policy imposes colossal computational requirements (compute-bound), performing a multi-step denoising cycle by the neural network on each control cycle makes real-time deployment of the model virtually impossible on embedded edge platforms with limited memory bandwidth.

The limitations of ACT (packet discreteness) and Diffusion Policy (computational and denoising redundancy) necessitated a transition to Flow Matching architectures, where the neural network predicts the velocity vector field in a single pass, and temporal smoothing is factored into deterministic mathematical functions (RTC). This is implemented in the `rtc.py` code of the LeRobot framework:

```
"v_t = original_denoise_step_partial(x_t)
    x_t.requires_grad_(True)
x1_t = x_t - time * v_t
err = (prev_chunk_left_over - x1_t) * weights
grad_outputs = err.clone().detach()
correction = torch.autograd.grad(x1_t, x_t, grad_outputs, retain_graph=False)[0]" [11]
```

Where the script accesses the trained neural network, and the model outputs the velocity vector  $\mathbf{v}_t$ . The current trajectory state container  $x_t$  is set to the gradient tracking flag (*True*) to calculate the tensor's influence on the final error.  $x1_t = x_t - time * v_t$  is a direct discretization of the equation  $dz_t = u_t dt$ . The algorithm takes a virtual step forward in time. It calculates the coordinate  $x1_t$ —that is, the location of the robot's actuators at the next microstep.

*prev\_chunk\_left\_over* — the remainder of the old movement plan that the robot started executing a moment ago, but has not yet completed.

$x1_t$  is a virtual point of the new plan.

*weights* is an attention mask (temporal filter) that requires strict matching of trajectories in the first 5–10 steps of movement.

`torch.autograd.grad` function calculates the partial derivative (Jacobian) of the predicted point  $x1_t$  with respect to the current state  $x_t$ , weighted by the recorded error *grad\_outputs* at index *[0]*, i.e., the gradient tensor itself. The algorithm calculates the mathematical force vector, the precise angle and direction each servo must be rotated to ensure the new trajectory smoothly flows from the old one without losing the final goal.

### 1.3.2. The problem of generalization in the open world.

A fundamental barrier to deploying AI systems on physical devices in unstructured environments is ensuring the ability to generalize perceptual-motor skills to new scene configurations not encountered in the training dataset. In this study, the task is formulated as precision manipulation (grasping and transferring objects) in a variable environment. In field experiments with the SO-ARM101 manipulator, the generalization problem is decomposed into three independent domains:

- Spatial Generalization: The system must be invariant to target displacement and rotation. The model must successfully generate a targeting trajectory.
- Visual Generalization: Resistance of the perceptual tract to changes in the illumination conditions of the work area, as well as to the appearance of background visual noise (foreign objects).
- Semantic Generalization: The ability of the SmoLM2 core to correctly interpret synonymous text instructions and associate them with specific features (color, shape) in the visual space of tokens.

When using standard imitation learning methods (behavior cloning), the system is extremely sensitive to the covariate shift phenomenon. The slightest error in the positioning of the physical actuator during the first step of the movement results in the gripper webcam capturing a scene angle in the next step that was not present in the record training set. In classical regression architectures, this causes compounding errors: the model enters an unfamiliar region of the latent space, generates anomalous control signals, and completely fails the task.

In the system under study, overcoming the problem of covariate shift and ensuring generalization in the open world is achieved through the synergy of two factors:

- Fundamental pretraining: Using the HuggingFace base weights *lerobot/smolvla\_base* [12] Trained on 23,000 episodes and approximately 500

community datasets. The model is inherently robust to background and perspective changes.

`torch.autograd.grad` gradient calculations, continuously smooths out delays and maintains the manipulator's physical trajectory within the continuous Flow Matching vector field calculated by the neural network, returning the trajectory in hardware despite any random external disturbances. Thus, calculating the Jacobian correction via *the torch.autograd.grad* operator at a frequency of tens of Hertz ensures the dynamic return of the actuator's trajectory to a stable state under any circumstances, guaranteeing the stability of spatial behavior.

### 1.3.3. Features of collecting datasets for VLA tasks.

The collection and structuring of naturalistic demonstrations for multimodal VLA-class control policies are subject to strict architectural criteria that differ from those of classical computer vision. Models based on the Transformer architecture possess fundamental redundancy and a "data-hungry nature" due to the absence of a priori inductive biases relative to the geometry of the physical world. To form stable spatial-semantic representations, any Transformer attention mechanism requires end-to-end pretraining on extremely large heterogeneous datasets. To overcome this barrier, the composite borrowing and Representation Transfer methodology is applied when assembling newly created multimodal architectures. Integration of the pretrained visual encoder ViT/SigLIP (dim=768) and the linguistic kernel SmolLM2 (dim=960) (Llama-like topology) into the SmolVLA perceptual-cognitive tract significantly simplifies and accelerates the training process for a new model. The borrowed blocks already possess a developed high-level semantics, invariant to scene conditions and command syntax. This allows the cognitive base to be completely isolated (frozen) from gradient calculations. The local dataset is collected in a highly specialized manner and is not used to train the model in basic concepts of vision or language, but solely for the targeted adaptation of the 16 layers of `lm_expert`, which simultaneously functions as a Language Model and Flow Matching Model.

The implementation of the new LeRobotDataset v3 standard completely restructures the perceptual-motor pipeline, optimizing input/output (I/O) during the registration and preparation of field data. Unlike earlier standards, where each demo episode was recorded in an isolated file, the v3 specification uses end-to-end concatenation. Multiple episodes are continuously packaged into coarse-grained data files (Apache Parquet for signals and MP4 for video sequences). This dramatically reduces the load on the OS file system and accelerates the operation of PyTorch batch loaders. Extraction of specific trajectories is performed via relational metadata meta */episodes/*, which stores precise byte and frame offsets. By implementing the

StreamingLeRobotDataset class, model pretraining can be performed in real time directly from Hugging Face Hub repositories without the need to download and deploy gigabyte arrays to the host's local disk (Hub-native Streaming). This allows the motor expert to dynamically consume distributed cross-robotics experience on the fly, optimizing memory usage. In the v3 standard, perceptual transformations (brightness (ColorJitter), contrast, and sharpness (SharpnessJitter) are completely isolated from the raw dataset. Camera frames are captured in their original physical form, and pixel masking and distortion to improve the robustness of the ViT encoder are applied exclusively during training, via the *ImageTransformsConfig* configuration. *This prevents recompression artifacts and allows for multiple testing of different spatial generalization profiles on a single dataset. Using incremental writes to Parquet tables with buffering, the dataset.finalize ()* method must be called after collecting demos. Without finalizing the Parquet file metadata structure and footers, the generated local cache (including the recorded session) is considered corrupted, which completely blocks its deserialization in the batch loader.

## **2. Practical implementation and experiments in the LeRobot environment.**

### **2.1. LeRobot Software ecosystem.**

#### **2.1.1. Overview of the LeRobot Framework by Hugging Face.**

For the development of the SO-ARM101 manipulator control hardware and software system, the open-source LeRobot framework, developed by the Hugging Face robotics division and a community of independent researchers, was chosen as the basic software platform. The purpose of this library is to standardize, scale, and support the full development cycle of end-to-end simulation learning (EIL) systems and multimodal VLA-class control in the physical world. The architectural topology of the LeRobot ecosystem is based on three fundamental modules abstracted from specific hardware: data collection, policy assembly, and building, low-level work with actuators.

The LeRobotDataset v3 data acquisition and serialization framework provides unified mechanisms for high-speed buffering of live demonstrations. The data acquisition module asynchronously polls OpenCV sensor subsystems for RGB vision, RealSense SE2 for RGB-D, and the 1-Wire manipulator actuator bus and CAN buses. A key feature of the third version of the format is the end-to-end aggregation of metadata into Parquet tables of the shared pool and the decoupling of heavy video streams into MP4 binary chunks, which optimizes input/output (I/O) and eliminates frame loss during teleoperation.

The framework is built entirely on declarative programming principles using the Hydra configuration manager and the draccus parser. The model factory (make\_policy) allows for seamless switching between different robotics policy types (ACT, Diffusion Policy, SmolVLA Pi0, etc.), encapsulating the network's internal mathematics from the main executable script. DataLoaders support StreamingLeRobotDataset, a cloud-based streaming technology that allows for mixing local field-based experience with large remote pretraining datasets on the fly without memory overload.

A low-level hardware interface (Hardware Abstraction Layer — HAL), built-in drivers, and CLI calibration utilities (lerobot-calibrate) for direct communication with general-purpose servo drives (Feetech and Dynamixel buses) via the UART/Serial protocol. The HAL layer performs a two-way function: it translates raw encoder ticks into proprioceptive state vector radians (dim=32) and receives predictive velocity packets (Action Chunks) from the Flow Matching (RTC) output stage, distributing them to the physical motor controllers.

Using the LeRobot framework eliminates the need to write drivers and thread synchronization modules, allowing us to focus on exploring the architecture of the monolithic SmolVLA decoder, the RTC math processor, and optimizing the pipeline and its components. However, when exploring the mechanisms of the vast majority of open - source projects, where each contributor considers it their "sacred duty" to "indelibly improve the project," documentation becomes outdated even while being written. Consequently, numerous problems must be addressed: extremely insufficient and inconsistent documentation; existential inconsistencies in the project structure and, consequently, module import paths; and conflicts between driver and library versions. The draccus-based argument parser is particularly noteworthy; consensus with it is often difficult to achieve using the *--help argument* or manual brute - force.



### 2.1.2. Environment configuration and SmolVLA integration.

According to the official LeRobot deployment guidelines, the framework authors recommend using isolated Conda environments via the Miniforge distributor to manage dependencies and ensure cross-platform policy portability. This step is critical for robotic systems, as standard package managers like pip and venv do not ensure the atomicity of low-level C++ extensions. For this study, the author implemented the recommended host configuration pipeline on an Ubuntu-based OS.

Using the miniforge toolkit, an isolated environment with a hard-wired Python 3.12 interpreter was deployed. To prevent Ubuntu system paths from interfering with the CUDA kernel linking, automatic activation of the Anaconda base context was forcibly disabled using the *conda config --set auto\_activate false* directive.

Integrating the SO-ARM101 manipulator and the SmolVLA policy required a composite build of the framework from source code (From Source). During the build phase, specialized meta-packages of dependencies, strictly defined by the manual, were activated:

```
git clone https://github.com/huggingface/lerobot.git
cd lerobot
pip install -e ".[all]" [13]
```

By default LeRobot architecture uses specialized TorchCodec library for high-speed Zero-copy decoding of MP4 video chunks directly V PyTorch tensors. The Hugging Face manual postulates that strict synchronization of FFmpeg codec versions is required for TorchCodec to function correctly. The standard package installation, *conda install ffmpeg*, deploys the latest experimental versions (FFmpeg 8.X), which causes terminal collisions with *torch\_dtype\_float4\_e2m1fn\_x2* and an OSError process crash when running with the stable PyTorch kernel. To resolve this version conflict, the author used a documented method of forcibly downgrading the multimedia backend to the industry standard FFmpeg v7.1.1 via a specialized channel, *conda install ffmpeg=7.1.1 -c conda-forge*.

### 2.1.3. Preparation and normalization of data (LeRobotDataset).

In the latest version of the LeRobot framework (v0.5.2), the structure and logic of preprocessing real-world demonstrations have been completely rebuilt to the LeRobotDataset v3.0 standard. The key architectural principle of the new specification is the complete separation of physical storage and the user application programming interface (API). This enables the conversion of raw signals from the SO-ARM101 manipulator and video camera streams into high-level normalized PyTorch tensors with minimal load on computing nodes.

Unlike legacy formats that operated with isolated files for each individual movement, batch aggregation and relational File-based Storage indexing are implemented. Low-dimensional, high-frequency signals (angles in radians, timestamps) are continuously concatenated in time and written to coarse-grained Apache Parquet files (*data/*). Visual data is similarly stitched into sharded MP4 packets (*videos/*). Extraction of specific time windows for training the monolithic SmolVLA policy is performed not by file boundaries, but programmatically through a relational metadata layer (*meta/episodes/*), which stores precise frame and byte offsets. This optimization dramatically reduces the pressure on the OS file system, ensuring instant initialization of PyTorch batch loaders.

*the dataset.finalize ()* method after finishing a session recording is critical. This function forcibly flushes the metadata cache accumulated in RAM to disk, closes MP4 containers, and, most importantly, writes the Parquet table index footers. Without finalization, files are considered corrupted, and the built-in pyarrow engine completely blocks their deserialization during the training computational graph deployment stage.

Before feeding into the cognitive core, all low-dimensional state features (servo angles) undergo mandatory Z-score scaling. Statistical parameters of bias and variance are not calculated on the fly (which would cause GPU downtime), but are read from the static manifest *meta/stats.json* precomputed during finalization. The

framework converts radian ranges to a single variance based on the global means and standard deviations of the dataset, and video stream pixels are normalized using SigLIP weighting factors.

To ensure robustness and prevent open-world covariate shift, task demonstration collection was performed in strict accordance with the official Hugging Face data quality guidelines :

- Multi-view: The circuit is synchronized at 15 FPS and includes two channels standardized by nomenclature: egocentric observation.images.wrist (local correction) and allocentric observation.images.front (global view on a tripod).

- Visual Clarity: Frames are captured in neutral, stable lighting conditions, and the leader arm (so101\_leader) is intentionally positioned out of view, ensuring that the only moving objects in the frame are the follower arm and the target objects.

- Semantic density: Text annotations of tasks are strictly limited to concise, deterministic phrases without abstract indices (25 to 50 characters in length), which is critical for the correct operation of the *lm\_expert linguistic-motor* expert's cross-attention mechanisms. The ImageTransformsConfig configuration used during the DataLoader stage imposes random perceptual distortions (ColorJitter, SharpnessJitter) exclusively in RAM during loading; the raw dataset on disk remains initially clean, eliminating video recompression artifacts and enabling infinite scaling of the model's spatial generalization profiles.

For the general case of recording a dataset, the author uses a recorder call with the following set of arguments :

```
lerobot-record \  
-- robot.type =so101_follower \ # type robot  
-- robot.port =/dev/ttyACM0 \ # port  
--robot.id=my_awesome_follower_arm \ # name file calibration  
-- robot.cameras ="{  
wrist: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30, warmup_s: 5},  
top: {type: intelrealsense, serial_number_or_name: 918512072179, width: 640, height: 480, fps: 30 } } "  
-- teleop.type =so101_leader \  
-- teleop.port =/dev/ttyUSB0 \  
--teleop.id=my_awesome_leader_arm \  
--display_data=true \ # opens a GUI with current telemetry (angle graphs and video stream)  
-- dataset.repo_id=StasGT/record-test \ # save repository  
-- dataset.push_to_hub=true \ # push on hub  
-- dataset.private =true \ # make the dataset private  
--dataset.num_episodes=50 \ # total number of episodes  
-- dataset.episode_time_s=60 \ # maximum episode time  
-- dataset.reset_time_s=10 \ # reboot time to prepare the scene for recording a new episode  
-- dataset.single_task="Instruction..." # Command robot (language context)
```

## 2.2. Setting up the experiment.

The operation of spatially selectively capturing and moving a small household object according to text instructions was chosen as a reference real-life task for verifying the hardware-software complex and the monolithic VLA policy SmolVLAPolicy. The semantic content of the target command is formalized as a deterministic string variable: "Put the boxes into the container". This task is classified in modern robotics as a complex perceptual-motor operation with sensory uncertainty of the work zone in an unstructured environment, without a clear physical-kinematic execution scenario.

The target object is located on a flat horizontal surface within the reach of the SO-ARM101 manipulator. The local scene is intentionally not isolated from external disturbances. Elements of unstructured chaos are introduced into the field of view of the static camera (top) and the egocentric gripper sensor (wrist): random variations in the intensity of natural light in the work area, shadows from the robot's structural elements, and background visual noise in the form of non-target objects of similar geometry. This precludes the use of primitive color thresholding algorithms for segmentation and requires the model to employ deep feature extraction.

The complete cycle of the manipulation task can be divided into three phases generated by the Flow Matching output stage in a single continuous flow, such as linguistic macro- guidance, visual accompaniment and terminal execution.

Thus, the robot reads the text trigger and, based on global features of the allocentric (stationary) camera (top), executes high-speed movement of the manipulator from the base home position to the vicinity of the target zone. As it approaches the object, when the robot's body occludes the view of the external camera, control switches entirely to the egocentric wrist channel. The model uses optical flow to precisely center the gripper axis normal to the longitudinal axis of the object, then detects when zero relative velocity is reached, opens the gripper, closes the servo, and then lifts the object.

The mathematical criterion of success (Success Rate) of task execution is the ability of the end-to-end conveyor to form a continuous linguo-diffusion field of velocities  $dz_t = u_t dt$  and process it through the RTC processor at a frequency of 40 Hz, ensuring the movement of the object by the manipulator in 10 control full-scale tests, under conditions of uncertainty of the initial position of the object.

### 2.2.1. Hardware setup and robot configuration (Leader-Follower setup).

The experimental setup is based on a two-manipulator "Master-Follower " design, recognized as the most effective method for high-fidelity simulation training. The setup includes two identical 5-axis SO-ARM101 manipulators, developed by TheRobotStudio, with identical kinematic structure.

The Leader Manipulator (so101\_leader) is used by the operator as a physical input interface (kinesthetic telecontrol). The torque-holding circuit is disabled on this device. The operator manually moves the leader robot's links along the target trajectory, while the integrated high-precision magnetic encoders of the servo drives record the angular movements of the joints in real time.

The Follower Manipulator (so101\_follower) is an executive natural organ that, at the recording stage, mirrors the leader's movements, and at the inference stage, processes predictive action packages (Action Chunks) generated by the SmolVLA model.

Both manipulators are equipped with specialized programmable Feetech servos connected via a UART half- duplex asynchronous serial data bus. The follower manipulator communicates with the control node via a USB-to-UART interface (FTDI/CH340 chip), registered in Ubuntu OS as the `/dev/ttyACM0` device. The data exchange rate is fixed at 1,000,000 baud rate, which minimizes delays during cyclic polling of motor registers. The leader manipulator is connected via an independent USB-to-Serial adapter to the `/dev/ttyUSB1` port at the same bus frequency. The author performed the proprioceptive alignment procedure (Kinematic Alignment) using the framework's CLI utility, *lerobot-calibrate*. During the step-by-step alignment protocol, individual zero offsets (`drive_offsets`) and hardware rotation limits of the magnetic encoders were determined for each of the six joints of both robot arms. The generated unique calibration profiles were automatically exported by the utility to the local environment cache:

(`~/.cache/huggingface/lerobot/calibration/`). During the data collection phase, the

framework's HAL layer loads these tables, instantly performing a linear transformation of the raw digital motor ticks into dimensionless normalized floating-point values (normalized floats) in the range  $[-1.0; 1.0]$ . This ensures that the 32-dimensional state vector reflects strictly identical spatial positions of the axes of both manipulators. The sensor subsystem deployed on the test bench is tightly synchronized with the recording utility at a frequency of 15 FPS and includes two high-resolution cameras (640x480 pixels):

- Egocentric channel (wrist): A monocular USB webcam integrated into the `OpenCVCamera` class, mounted on a movable plastic flange of the follower gripper.
- Allocentric channel (top): A static Intel RealSense 435i camera mounted on an independent tripod 40 cm from the work area and providing a frontal view of the entire table plane.

It should be emphasized that the standard architecture of the LeRobot repository completely lacks the functionality of geometric image dedistortion, which is due to the strict orientation of the framework authors on end-to-end training methods based on raw pixels.

In the Intel RealSense 435i, dedistortion and spatial alignment of the video stream are performed natively at the hardware level (Hardware-Based Rectification) via the integrated Intel RealSense Vision Processor D4 ASIC. The chip reads calibration parameters from the sensor's integrated EEPROM memory and performs streaming pixel remapping directly on the device's crystal before sending the data over the USB 3.0 bus. Since the camera has already been calibrated, no further actions are required and no additional solutions are needed.

To adapt the system to the optical distortions of low-end USB webcams, the author integrated the `cv2.remap` algorithm directly into the *read()* method of the `OpenCVCamera` class (`/src/lerobot/cameras`). To eliminate data flow downtime and minimize the load on processor cores, the calculation of the distortion matrices was moved out of the main loop and performed once via the `cv2.initUndistortRectifyMap` operator during sensor initialization. High-speed linear remapping at each acquisition



step ensures topologically correct visual features are fed to the monolithic SmolVLA decoder, fully compensating for the hardware limitations of sensors without changing the framework's basic training cycle.

### 2.2.2. Data collection protocol and demonstration recording methodology.

The experimental stage of generating a local set of demonstrations for adapting the monolithic linguomotor expert `lm_expert` was performed under strictly controlled conditions according to a defined step-by-step protocol. The target task was formulated as the physical capture and movement of objects. To initialize the data collection session, the author used the framework's global CLI utility with the following attributes:

```
lerobot-record \
-- robot.type =so101_follower \
-- robot.port =/dev/ttyACM0 \
--robot.id=my_awesome_follower_arm \
-- robot.cameras ="{
wrist: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30, warmup_s: 5},
top: {type: intelrealsense, serial_number_or_name: 918512072179, width: 640, height: 480, fps: 30,
use_depth: True } }" \
-- teleop.type =so101_leader \
-- teleop.port =/dev/ttyUSB0 \
--teleop.id=my_awesome_leader_arm \
--display_data=true \
-- dataset.repo _id=StasGT/record-put\
-- dataset.push_to_hub=true \
-- dataset.private =false \
--dataset.num_episodes=50 \
-- dataset.episode_time_s=60 \
-- dataset.reset_time_s=10 \
-- dataset.single_task="Put the boxes into the container"
```

Visual data comes from two cameras: a USB webcam on the manipulator gripper and an Intel RealSense D435i for RGB and depth data capture. The project developers did not implement depth maps, so the author of this paper modified the project code. The `get_observation()` method of the *SOFollower* class additionally implements depth map retrieval, processing, normalization (raw data has a 12-bit format), and addition of the data to the final observation dictionary *obs\_dict*, which contains camera frames and the states of each servo.

Next, 50 demonstration episodes were executed, each with a maximum duration of 60 seconds and a 10-second runtime reboot. The frame rate was unlimited

and was 30 FPS for both views; no frame losses or drive states on the asynchronous UART bus (1 Mbod) were detected. Each episode included variable task implementation scenarios necessary to improve the generalization capacity of the network. Before the start of each episode, the author arbitrarily shifted objects along the table plane within a radius of up to 150 mm from the central point, simulating an unstructured environment. The master (so101\_leader) smoothly moved, directing the actuator (so101\_follower) from the home position to the object, intentionally varying the speed and curvature of the macro-trajectory to enrich the flow-matching velocity fields. The gripper was precisely guided along the normal to the longitudinal axis of the object, closing and lifting the object. After all episodes completed, the framework core automatically executed the *dataset.finalize ()*. The method generated relational metadata structures in the meta/episodes/ directory, calculated constant shift coefficients and feature variances in the meta/stats.json manifest, and, most importantly, wrote index footers for the Apache Parquet tables. This approach ensures the absolute integrity of the collected dataset and correct step-by-step deserialization by the PyTorch loader during the training phase.

### 2.2.3. Methodology and program regulations for model fine-tuning.

The *lm\_expert* linguistic-motor expert and adapters, and the low-rank *LoRA adaptation method of the peft* toolkit. Based on a full-scale computational experiment conducted on a host with an NVIDIA RTX 2060 Super (8GB), 64GB DDR-5, and a 16-core Intel Core i7-13700KF graphics accelerator, using the standard CLI, the following hyperparameter profile was generated and verified:

```
lerobot-train \
-- policy.type =smolvla \
-- policy.pretrained_path=lerobot/smolvla_base \
-- dataset.repo_id=StasGT/record-put \
-- dataset.root =/home/stas/.cache/huggingface/lerobot/StasGT/record-put_20260505_171305 \
-- policy.repo_id=StasGT/SmolVLA-Instruct_put \
--output_dir=outputs/train/SmolVLA-Instruct_put \
--job_name=SmolVLA-Instruct_put \
-- policy.device =cuda \
-- policy.optimizer_lr=1e-4 \
-- policy.scheduler_decay_lr =1e-5 \
--steps=10000 \
--save_checkpoint=true \
--save_freq=500 \
--batch_size=24 \
```

The mathematical formalization and numerical values of the hyperparameters of the SmolVLA policy retraining process were determined by the author experimentally within the constraints of the target computing node.

Scaling the batch to 24 elements fully utilized the registers of the Turing architecture's tensor cores, ensuring effective smoothing of stochastic gradient noise. Selecting a value that is a multiple of the internal computational topology transferred the problem to the class of compute-bound processes, stabilizing video memory utilization at a safe level of 7.0 GB. The iteration profile size was fixed at 10,000 steps. With the achieved computational rate of 3.2 seconds per step, the total time spent on one full training cycle was 8.5 hours, which allowed the VLA policy training to be successfully completed within a single night cycle. During this period, the expert's monolithic linguistic-motor core processed  $10,000 * 24 = 240,000$  unique

multimodal states of the SO-ARM101 manipulator, ensuring deep convergence of the objective function.

Based on the author's software audit of the PyTorch active-memory gradient field, the optimization step was fixed at a moderate, balanced level -- *policy.optimizer\_lr = 1e-4*, located on the border between the extremely aggressive  $1e-3$  and conservative  $1e-5$  gradient descent strategies. This compromise is scientifically justified by the identified topology of the default policy parameter distribution, within which 16% of the total parametric capacity of the graph (72,414,032) of (450,000,000) parameters are in training mode. Isolation from gradient calculations of the ViT/SigLIP core perceptual blocks (*freeze\_vision\_encoder=True*) and SmolLM2 self-attention layers ensures that all high-level semantics of vision, text context, and attention remain unchanged throughout all 10,000 iterations. Optimization is performed exclusively on intermodal interface bridges (*model.state\_proj*) and internal projectors of the MLP components of each of the 16 levels of the linguomotor block *lm\_expert* (*gate\_proj*, *up\_proj*, *down\_proj*). *With this step, AdamW's gradient descent performs targeted, high-precision scaling and reweighting of cognitive data.*

In this study, as mentioned earlier, an additional modal perception channel—metric depth maps—is introduced into the perceptual tract of the SmolVLA policy as a fundamental scientific and practical innovation. The introduction of a third-dimensional spatial feature required local, mathematically isolated architectural changes to the computational graph of the model. In the author's earlier study [14], an attempt at deep structural deconstruction was made: replacing the basic ViT encoder of the multimodal BLIP model with a heterogeneous encoder-decoder architecture DeTr [15]. This maneuver, despite a local increase in computational performance by 46%, led to terminal collapse and a complete loss of the semantics of the linguistic block [14]. The model exhibited the classic overfitting effect: it successfully approximated only within the training set, but completely lost the ability to semantically generalize on validation tests, since the negligible size of the local

dataset did not allow the transformer to relearn the syntax of the world. Taking into account previously identified patterns [14], in this paper, the author implemented a method of asymmetric cross-modal induction without violating the invariance of the underlying distributions. A custom cross-attention bridge developed by the author was integrated into the prefix tokenization circuit at the junction of the frozen ViT/SigLIP encoder and 16 layers of the SmolLM linguomotor planner. The mathematical graph of the algorithm projects highly semantic queries (Q) of RGB vision tokens onto trainable matrices of keys (K) and values (V) of geometric distance tokens formed by a parallel lightweight CNN of depth. This approach allowed us to enrich the prefix with third-dimensional features without degrading the original network's cognitive performance. Through a series of real-world computational experiments using the LeRobot framework, the author obtained and verified three comparative policy retraining profiles:

- Basic Adaptation Profile (**Default Configuration**): Implements selective Full Parameter Tuning exclusively within the MLP components of the `lm_expert` expert with a hard freeze of 83.91% of the network. Software audits recorded optimization of exactly 16.09% of active weights (72,414,032 parameters), which ensured peak memory usage remained at 7.0 GB of VRAM per data batch with `batch_size = 24` and a stable computational speed of 3.2 seconds per iteration.

- Low-Rank Perceptual Adaptation Profile (**LoRA-ViT**): Incorporating low-rank LoRA adaptors ( $r=8$ ) into the attention layers of the ViT/SigLIP encoder. This approach ensured local alignment of visual features under new conditions, but resulted in an 18% increase in optimization step latency due to the overhead of calculating gradient backpass deltas.

- Extended Perception Profile (**RGB-D- Extended**)[16], [17]: Configuration with an activated custom Cross-Attention bridge at the input of the linguomotor unit. Restoration and retention of the canonical latent dimension at the 960-feature level is implemented natively at the linear algebra level of the cross-attention unit itself—by projecting *Query (Q) vectors* from the original 960-dimensional SigLIP space onto

*the Key (K) and Value (V) matrices* of the depth map. Additional training of the MLP layers of the expert projectors demonstrated monotonic convergence of the loss function, natively training a closed-loop model taking into account the physical distance to the object in millimeters while fully preserving the accumulated linguistic intelligence of the foundation network.

## 2.3. Analysis of results.

### 2.3.1. Metrics (Success Rate).

To quantify the effectiveness of the developed architectural solutions and verify the trained SmolVLAPolicy policy, the success rate (SR) was chosen as the key integral metric. This parameter is defined as the ratio of the number of successful natural manipulations to the total number of experimental attempts:

$$\mathbf{SR} = \mathbf{N\_success} / \mathbf{N\_total} * 100\%$$

Experimental testing was conducted in an unstructured environment for the target command "*Put the boxes into the container.*" A series of 10 isolated test runs was conducted for each of the three generated retraining profiles (the standard RGB pipeline, LoRA-ViT, and the developed RGB-D Cross-Attention circuit). Before each test run, the target object (box) was randomly displaced along the tabletop by up to 15 cm relative to the central demonstration point and also rotated through a random angle.

A manipulation cycle was considered completely successful ( $N_{\text{success}}$ ) only after the entire task, consisting of the subtasks of gripper guidance, grasping, moving, and placing the object in the specified space, was completed.

During testing, the critical impact of the Real-Time Chunking (RTC) mathematical circuit on the resulting SR metric was experimentally recorded. When the `rtc.py` script was disabled, the model's inference dropped due to delays, leading to discrete jerks in the drives and a 20–30% drop in the SR metric due to electrical shocks and spatial spread of the axes. The integration of the RTC processor, which calculates the correction gradients `torch.autograd.grad` at a frequency of 40 Hz, enabled seamless stitching of predictive velocity packets (Action Chunks), guaranteeing continuous motion.

The RGB-base profile (with 16.09% weights retrained) demonstrated **SR = 70%** (7 out of 10 successful acquisitions). Errors were caused by shadow fluctuations, which distorted the range estimate from a flat image.



The LoRA-ViT profile showed metric degradation to **SR = 60%**, likely due to partial semantic loss and local overfitting, or statistical error. The author believes it appropriate to consider this method unpromising.

RGB-D-Extended The extended architecture ( $RGB(Q), Depth(K, V) \rightarrow Cross-Attention$ ) demonstrated the highest robustness, recording **SR = 90%** (9 out of 10 successful manipulations). Asymmetric targeting of SigLIP tokens to the key-value matrices of the Intel RealSense D435i depth map completely eliminated geometric uncertainty in the scene, ensuring precise closed-loop positioning of the manipulator's actuator throughout the entire workspace.

This study does not present the test data for the basic pre-trained model *lerobot/smolvla\_base* [12], since they give a zero result, regardless of the control command, the robot's servos rush to a position equal to "0", i.e., each joint ends up in its average position, and the gripper is in the "closed" position. This mathematical phenomenon is well known in deep learning and probability theory as *Regression to the Mean* in MSE optimization [18], and in robotics and imitation learning (Behavior Cloning) it often manifests itself as *The Blurring Problem / Mean Prediction Effect* [19]. In other words, under conditions of high entropy of features, uncorrelated vectors of target positions of actuators, symmetrically distributed in the normalized space [-1.0; 1.0], are forcibly approximated by the network to their mathematical expectation, equal to zero mean = 0.

### 2.3.2. Evaluation of the stability of the quantized model for embedded processors with NPU.

The choice of peripheral device with an NPU was made from what the author had on hand. The most suitable device turned out to be the Orange Pi AI Pro microcomputer (HUAWEI Ascend quad - core 64-bit ARM CPU (GPU + NPU 8 TOPS/ Int 8 (DaVinci AI processor), 16GB LPDDR4X). Transferring the compiled SmolVLAPolicy policy graph from the high-performance host context to the edge embedded processor is limited by the hard bandwidth of the LPDDR4X memory and the processing power of the quad-core ARM64 processor. To ensure real-time inference of transformer networks at the data collection frequency, hardware graph compilation and integer quantization (Post-Training Quantization) are mandatory.

Since the board's target neural network processing unit (NPU) is based on the Huawei Ascend architecture, the intermediate ONNX graph, exported with rigidly fixed input masks (Static Shapes), was translated into the final hardware-dependent binary *.om* file using the *Ascend Tensor Compiler utility*. During compilation, integer quantization to the INT 8 level was implemented layer-by-layer for all weight matrices. Both the entire SmolVLM model and the *RGB-D-Extended bridge developed by the author were quantized*. To minimize quantization error and prevent activation scale mismatches, a calibration dataset consisting of 30 representative multimodal prefixes was used. Experimental analysis of the inference of the quantized *.om* graph on an Orange Pi AI Pro revealed the high resilience of the author's architecture to reduced weight bit depth.

Due to the high redundancy of the 16 expert layers, the accuracy drop in Flow Matching velocity vector approximation was no more than 2.3% compared to the baseline float32 graph on the host GPU. Integer compression of the 450 million modified contour parameters to INT8 resulted in a 4-fold reduction in RAM footprint (410 MB) and dramatically reduced the load on the LPDDR4X bus. The peak inference frequency of the entire VLA policy directly on the NPU chip stabilized at

6.4 Hz (the generation time of one Action Chunk is 150 ms). Despite the lag in the NPU neural network's inference frequency compared to the Feetech servo's physical sampling loop (40 Hz), control stability was fully compensated by the CPU-level Real-Time Chunking algorithm, which provides precise closed-loop smoothing of phase shifts and trajectory matching during the initial steps of the new manipulator motion prediction. This completely dampened the NPU hardware lag and ensured that the final success rate remained at **SR = 90%**, demonstrating the high stability and applicability of the quantized system.

## Conclusion.

As part of the study, an end-to-end hardware and software system for multimodal VLA control was experimentally verified based on the low-parameter SmolVLA (0.45B) foundational architecture, integrated with the SO-ARM101 5-axis manipulator. The low-parameter SmolVLA architecture under study offers enormous technological potential for deployment onboard autonomous mobile robots.

The 450 million parameter capacity allows the model to be deployed directly on ultra -small devices (Orange Pi AI Pro, Jetson Orin Nano), completely eliminating API latency and ensuring robot fault tolerance regardless of connectivity.

By inheriting pre-trained SigLIP and SmolLM2 models, the mobile robot gains the ability to flexibly interpret user text instructions in unstructured warehouse or home environments, performing on-the-fly object feature generalization without the need for rigorous retraining. The power consumption of the I NT 8-quantized graph on the embedded board is less than 15 W. This is critical, as it minimizes the power consumption of the onboard batteries, extending battery life.

Despite the achieved field test success rate ( $SR = 90$ ), a thorough technical audit revealed a number of fundamental limitations of the current implementation:

The peak inference frequency of the neural network on the Orange Pi AI Pro NPU is fixed at approximately 6.5 Hz. This limitation is caused by the bottleneck of the LPDDR4X memory bus during autoregressive reading of transformer layer weights. A frequency of 6.5 Hz is critically insufficient for highly dynamic AMRs (e.g., walking robots), which require a control signal update rate of at least 100 Hz. Although the Real-Time Chunking mathematical processor successfully smooths out phase shifts, if the environment changes abruptly, the model will not have time to instantly correct its behavior, and the robot will continue to process the old Action Chunk calculated before the scene change.

To overcome the identified limitations, the following engineering solutions are planned for implementation in future research and development work:

- Dividing the model into sub-models with parallelization of computations.
- Replacing SigLIP with an efficient hybrid CNN + Attention encoder.
- Implementation of a model of semantic division of a command into actions.
- Implementation of speculative decoding of actions.
- Implementation of Mixture of Experts (MoE).
- Dynamic quantization.

## References

1. Dosovitsky A ., Beyer L ., Kolesnikov A . et al . An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale //arXiv:2010.11929 – 2020.
2. Zhai X., Mustafa B., Kolesnikov A. et al. Sigmoid Loss for Language Image Pre-Training //arXiv:2303.15343 – 2023.
3. Loubna B. et al. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model //arXiv:2502.02737 – 2025.
4. Shukor M., Aubakirova D. et al. SmolVLA: A vision-language-action model for affordable and efficient robotics //arXiv:2506.01844 – 2025.
5. Lipman Y., Chen R. et al. Flow matching for generative modeling //arXiv:2210.02747 – 2022.
6. Chen R., Rubanova Y. et al. Neural Ordinary Differential Equations //arXiv:1806.07366 – 2019.
7. Black K., Galliker M., Levine S. Real-Time Execution of Action Chunking Flow Policies //arXiv:2506.07339 – 2025.
8. Zhao T., Kumar V, Levine S., Finn C. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware //arXiv:2304.13705 – 2023.
9. Kingma D., Welling M. Auto-Encoding Variational Bayes //arXiv:1312.6114 – 2013.
10. Vaswani A., Shazeer N. Attention Is All You Need //arXiv:1706.03762 – 2017.
11. LeRobot //GitHub repository, 2025. URL: <https://github.com/huggingface/lerobot>
12. SmolVLA\_base (LeRobot) weights //Hugging Face repository, 2025. URL: [https://huggingface.co/lerobot/smolvla\\_base](https://huggingface.co/lerobot/smolvla_base)
13. Hugging Face LeRobot Docs // URL: <https://huggingface.co/docs/lerobot>

14. Trenikhin S. Research of the transformer architectures and fine-tuning in the task of generating text descriptions //Report at the HSE University in Nizhny Novgorod – 2025.
15. Carion N., Massa F., Kirillov A. et al. End-to-End Object Detection with Transformers //arXiv:20 05.12872 – 2020.
16. SmolVLA-RGB-D-Extended //GitHub repository, 2026. URL: <https://github.com/StasGT/SmolVLA-RGB-D-Extended>.
17. SmolVLA-RGB-D-Extended weights //Hugging Face repository, 2026. URL: <https://huggingface.co/StasGT/SmolVLA-RGB-D-Extended>.
18. Bishop CM Pattern Recognition and Machine Learning. – Springer, 2006.
19. Chi C. et al. Diffusion Policy: Visuomotor Policy Learning through Action Diffusion // Proceedings of the Topic Robotics: Science and Systems (RSS). – 2023.